



المدرسة العليا للتكنولوجيا
جامعة الحسن الثاني الدار البيضاء
Ecole Supérieure de Technologie
Université Hassan II de Casablanca

Ouvrage pédagogique

Développement web avec Django

Conception et développement d'applications web

M'barek IAOUSSE

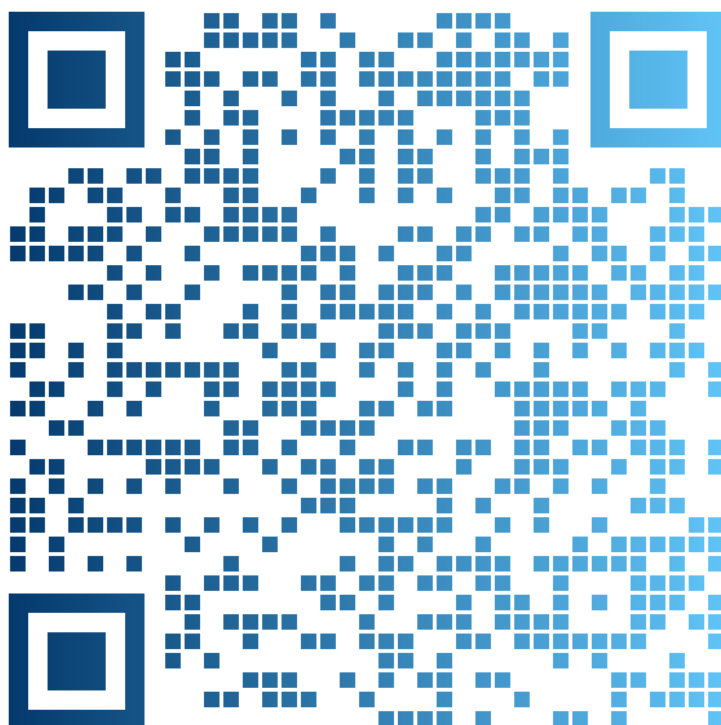
Formation continue ASERCS

École Supérieure de Technologie de Casablanca

Université Hassan II de Casablanca

Première édition — 2026

Version en ligne



Scannez ce code pour accéder à la version en ligne.

Cet ouvrage pédagogique a été élaboré comme support du cours *Développement web avec Django*, proposé dans le cadre de la **Formation continue ASERCS** à l'École Supérieure de Technologie de Casablanca (ESTC).

L'auteur : M'barek IAOUSSE, Enseignant-Chercheur à l'EST Casablanca, Université Hassan II de Casablanca.



Sauf indications contraires, le contenu de ce manuel électronique est disponible en vertu des termes de la [Licence Creative Commons Attribution 4.0 International](#).

Autorisations accordées :

Partager – copier, distribuer et communiquer le matériel par tous moyens et sous tous formats.

Adapter – remixer, transformer et créer à partir du matériel pour toute utilisation, y compris commerciale.

Sous réserve de citer l'auteur.

© M'barek IAOUSSE (2026)

Pour citer ce manuel : M'barek IAOUSSE. (2026). **Développement web avec Django**. École Supérieure de Technologie de Casablanca. Sous licence CC BY 4.0.

Préface

Le présent ouvrage est le support du cours **Développement web avec Django**, proposé dans le cadre de la **Formation continue ASERCS** à l'École Supérieure de Technologie de Casablanca (ESTC).

Ce cours s'adresse aux personnes qui possèdent des bases en programmation Python et souhaitent apprendre à construire des applications web. Une connaissance préalable de Django n'est pas nécessaire.

Le parcours accompagne pas à pas la création de **BiblioCours**, une application de gestion et de consultation de cours, depuis la mise en place du projet jusqu'à sa préparation pour la production.

Le manuel présente l'architecture d'une application Django, le routage des requêtes, les modèles et l'ORM, les migrations, les vues et templates, les formulaires et l'administration, puis l'authentification, les pratiques de sécurité, les tests et le déploiement. L'objectif est de développer une démarche rigoureuse pour concevoir des applications web maintenables et fiables.

Prérequis

Le contenu de ce manuel est organisé pour des personnes ayant déjà une base en :

- la programmation en Python (variables, conditions, boucles, fonctions et structures de données de base) ;
- le HTML et les notions élémentaires du fonctionnement du Web.

Une expérience préalable avec Django n'est pas requise : les notions nécessaires sont introduites progressivement et mises en pratique tout au long du cours.

Ressources complémentaires

La documentation officielle de [Django](#) constitue la ressource de référence pour approfondir les fonctionnalités du framework et consulter les informations correspondant à chaque version.

Les exemples et exercices pratiques de ce manuel sont développés en Python avec Django. Ils suivent la réalisation d'une même application afin de relier les concepts aux étapes concrètes du développement web.

Note

Les versions de Python et de Django évoluent. Suivez les versions et les consignes d'installation précisées dans le chapitre de démarrage, et consultez la documentation officielle en cas de différence avec votre environnement.

Sommaire

Préface	i
Démarrer proprement avec Django et uv	1
Ce que nous allons construire	1
Pourquoi Django ?	1
Une requête, concrètement	1
Pourquoi uv remplace <code>venv</code> et <code>pip</code> dans ce cours	2
Installation et vérification	2
Initialiser le projet	2
Ajouter Django	3
Créer le projet Django	3
Le rôle de chaque fichier	4
Créer l'application <code>cours</code>	4
Versionner ce qui doit l'être	5
Checkpoint	5
Exercices	6
Comprendre l'architecture Django et le routage	7
Le chemin complet d'une requête	7
Les apps : découper par responsabilité	7
Lire <code>settings.py</code> plutôt que le copier	8
Le secret et le mode debug	9
Déclarer les premières routes	9
Écrire une vue minimale	9
Paramètres capturés dans l'URL	10
Comprendre la résolution des URLs	10
Requêtes et réponses : les distinctions importantes	11
Premier modèle mental à retenir	11
Exercice guidé	11
Modéliser les données avec l'ORM et les migrations	12
Penser d'abord au domaine	12
Écrire les modèles de BiblioCours	12
Pourquoi ces champs ?	13
Choisir <code>on_delete</code>	13
Les migrations : l'historique du schéma	14
Créer et lire des objets dans le shell	14
Filtrer, trier et limiter	15
<code>select_related</code> et <code>prefetch_related</code>	15
Modifier les données avec prudence	16
Concevoir avant de migrer	16
Exercices	16
Construire les pages : vues, URLs et templates	17
De la donnée à une page HTML	17
Les vues de liste et de détail	17
Pourquoi <code>render</code> ?	18
404 plutôt que page vide	18
Organiser les templates	19
Le template parent	19
Afficher une collection avec <code>liste.html</code>	19

Afficher le détail	21
Rechercher avec un paramètre GET	21
Lire un bug de template	22
Exercices	22
Saisir et administrer les données : formulaires et admin	23
Pourquoi un formulaire HTML ne suffit pas	23
Créer un <code>ModelForm</code>	23
Les trois niveaux de validation	24
La vue de création	24
Le template du formulaire	25
Modifier un objet existant	25
Supprimer : préférer une confirmation explicite	26
L'administration Django	26
L'admin et les performances	27
Messages utilisateur	27
Exercices	28
Authentification, sécurité et fichiers statiques	29
Authentifier n'est pas autoriser	29
Ajouter les routes d'authentification	29
Protéger une vue et une action	30
CSRF : protéger les requêtes qui changent l'état	31
Échappement, injection et entrées utilisateur	31
XSS et templates	31
Injection SQL	31
Redirections	32
Secrets et configuration	32
Fichiers statiques	32
Téléversements et fichiers média	33
Checklist de sécurité locale	33
Exercices	33
Tester, diagnostiquer et faire évoluer le code	34
Un test décrit un contrat	34
Lancer la suite	34
Tester le modèle	34
Tester les URLs et les vues	35
Tester l'authentification et les permissions	36
Tester un formulaire	37
Déboguer sans deviner	37
Qualité automatique avec <code>ruff</code>	38
Une petite stratégie de test	38
Exercices	38
Préparer la production et continuer l'apprentissage	39
Développement et production n'ont pas le même but	39
Séparer la configuration du code	39
Exécuter les vérifications Django	40
Base de données et migrations	40
Fichiers statiques	40
WSGI, ASGI et processus	41
Journalisation et erreurs	41
Déployer avec un <code>pyproject.toml</code>	42
Une checklist finale	42
Aller plus loin : une progression raisonnable	42
Projet de synthèse	43
Exercices de consolidation	43

Démarrer proprement avec Django et uv

Ce que nous allons construire

Ce cours suit la construction progressive de **BiblioCours**, une petite bibliothèque de cours en ligne. L'application permettra finalement de :

- classer des cours par matière ;
- afficher une liste puis le détail d'un cours ;
- gérer les données depuis l'administration Django ;
- permettre à des utilisateurs authentifiés de contribuer ;
- tester les comportements importants avant de les faire évoluer.

Le projet est volontairement assez petit pour rester lisible, mais assez réaliste pour faire apparaître les problèmes qui comptent dans une application web : organisation du code, données relationnelles, validation, sécurité, tests et déploiement.

Prérequis et version du cours

On suppose que vous savez lire du HTML et écrire des programmes Python simples (fonctions, classes, imports, exceptions et modules). Les exemples visent Django 5.2 et Python 3.12 ou une version plus récente compatible. Les idées restent valables pour les versions voisines, mais les détails de configuration doivent toujours être vérifiés dans la documentation de la version installée ([Django Software Foundation 2025b](#)).

Pourquoi Django ?

Une application web reçoit des requêtes HTTP, décide quoi faire, lit ou modifie des données, puis renvoie une réponse HTTP. On pourrait programmer chacune de ces opérations à la main. Django fournit plutôt une structure cohérente :

1. un routeur associe une URL à du code Python ;
2. une vue coordonne le traitement de la requête ;
3. un modèle décrit les données et dialogue avec la base ;
4. un moteur de templates produit le HTML ;
5. des composants prêts à l'emploi prennent en charge l'administration, l'authentification, les sessions et une partie des protections de sécurité.

Cette approche réduit le code répétitif, mais elle impose aussi des conventions. Apprendre Django revient donc moins à mémoriser des fonctions qu'à comprendre le rôle de chaque couche et le moment où elle doit intervenir. Le tutoriel officiel suit la même logique en construisant une application par étapes ([Django Software Foundation 2025j](#)).

Une requête, concrètement

Lorsque l'on ouvre `http://127.0.0.1:8000/cours/`, le navigateur envoie approximativement :

```
GET /cours/ HTTP/1.1
Host: 127.0.0.1:8000
Accept: text/html
```

Django reçoit cette requête, cherche une route correspondante, appelle la vue, éventuellement interroge la base de données, rend un template et renvoie par exemple :

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8

<!doctype html>
<html lang="fr">...</html>
```

Le code du cours rendra ce chemin observable. À chaque étape, nous chercherons à répondre à trois questions : **quelle est l'entrée ? quelle décision est prise ? quelle sortie est produite ?**

Pourquoi uv remplace venv et pip dans ce cours

Un environnement Python global est une mauvaise unité d'installation : deux projets peuvent exiger des versions incompatibles de Django ou d'une même dépendance. On veut isoler les dépendances du projet et pouvoir recréer exactement l'environnement sur une autre machine.

uv est un gestionnaire rapide de versions Python, d'environnements et de dépendances. Il utilise un fichier `pyproject.toml` pour déclarer le projet et un fichier `uv.lock` pour enregistrer la résolution exacte des dépendances ([Astral 2025](#)). Nous n'avons donc pas besoin de créer manuellement un dossier `env/`, ni de penser à activer un environnement à chaque terminal.

Installation et vérification

Installez uv en suivant la méthode officielle correspondant à votre système. Puis vérifiez que la commande est disponible :

```
uv --version
python --version
```

Sur Windows, PowerShell et Windows Terminal conviennent. Sur macOS et Linux, un terminal courant suffit. Si la commande uv n'est pas trouvée après l'installation, redémarrez le terminal afin que le PATH soit rechargé.

Initialiser le projet

Créez un dossier vide, puis initialisez un projet Python :

```
mkdir bibliocours
cd bibliocours

uv init --python 3.12
```

Cette commande crée notamment `pyproject.toml`. Le choix de Python doit être explicite : il évite qu'un collègue utilise par hasard une version très différente. Vous pouvez demander à uv d'installer une version si elle n'existe pas encore sur la machine :

```
uv python install 3.12
uv python pin 3.12
```

La commande `uv python pin` peut créer un fichier `.python-version`. Ce fichier documente la version attendue sans mettre Python lui-même dans Git.

Ajouter Django

Ajoutez Django comme dépendance de production :

```
uv add "Django>=5.2,<6"
```

Examinez ensuite les fichiers produits :

```
ls
cat pyproject.toml
head -30 uv.lock
```

Dans `pyproject.toml`, Django apparaît dans la liste des dépendances. Dans `uv.lock`, `uv` conserve les versions concrètement retenues, y compris les dépendances indirectes. Il faut versionner `pyproject.toml` et `uv.lock`; il ne faut pas versionner `.venv/`.

! `uv run` est le nouveau réflexe

Lancez une commande Python du projet avec `uv run` :

```
uv run python -c "import django; print(django.get_version())"
uv run django-admin --version
```

`uv run` vérifie l'environnement et exécute la commande avec les dépendances du projet. Vous n'avez donc pas à taper `source .venv/bin/activate`. Une activation manuelle reste possible, mais elle n'est plus nécessaire pour suivre ce cours.

Créer le projet Django

Dans Django, un **projet** représente la configuration globale d'un site. Une **application** est une unité fonctionnelle qui regroupe des modèles, vues, templates et tests. Un même projet peut réunir plusieurs applications : `cours`, `comptes`, `notifications`, etc. Cette distinction est importante : le projet assemble ; les applications portent les fonctionnalités.

Créons la configuration globale dans le dossier courant :

```
uv run django-admin startproject config .
```

Le point final signifie « utiliser le dossier courant » plutôt que créer un dossier imbriqué `config/config`. La structure initiale est :

```
bibliocours/  
  .python-version  
  pyproject.toml  
  uv.lock  
  manage.py  
  config/  
    __init__.py  
    asgi.py  
    settings.py  
    urls.py  
    wsgi.py
```

Le rôle de chaque fichier

- `manage.py` prépare l'accès aux réglages et expose les commandes Django ;
- `config/settings.py` contient la configuration globale ;
- `config/urls.py` est le routeur principal ;
- `config/asgi.py` et `config/wsgi.py` sont les points d'entrée utilisés par les serveurs applicatifs ;
- `__init__.py` indique que `config` est un paquet Python.

Lancez le serveur de développement :

```
uv run python manage.py runserver
```

Ouvrez `http://127.0.0.1:8000/`. Le serveur recharge souvent le code quand un fichier Python est modifié. Il est pratique pour apprendre, mais il ne doit pas être utilisé comme serveur public : il n'est ni dimensionné ni configuré pour la production.

Arrêtez-le avec `++ctrl+c++`. Une seconde commande dans un autre terminal peut inspecter le projet :

```
uv run python manage.py check
```

`check` analyse la configuration sans modifier la base de données. Prenez l'habitude de l'exécuter après une modification structurelle.

Créer l'application cours

```
uv run python manage.py startapp cours
```

La nouvelle structure ressemble à ceci :

```
cours/  
  __init__.py  
  admin.py  
  apps.py  
  migrations/  
    __init__.py  
  models.py  
  tests.py  
  views.py
```

Une app n'est pas active simplement parce que son dossier existe. Ajoutez-la dans `config/settings.py` :

```
INSTALLED_APPS = [
    "django.contrib.admin",
    "django.contrib.auth",
    "django.contrib.contenttypes",
    "django.contrib.sessions",
    "django.contrib.messages",
    "django.contrib.staticfiles",
    "cours",
]
```

Dans un projet plus grand, on préfère souvent `"cours.apps.CoursConfig"` : cette forme rend explicite la classe de configuration. La forme courte fonctionne ici car Django sait retrouver `CoursConfig`.

Versionner ce qui doit l'être

Initialisez Git et créez `.gitignore` :

```
git init
```

```
.venv/
__pycache__/
*.py[cod]
*.sqlite3
.env
.DS_Store
media/
staticfiles/
```

La base SQLite locale et les fichiers téléversés peuvent contenir des données de travail ; ils ne doivent pas être envoyés par défaut. Les secrets ne doivent jamais se trouver dans Git. `uv.lock`, en revanche, **doit** être conservé : il sert à reproduire l'installation.

```
git add .
git commit -m "chore: initialiser le projet Django"
git status
```

Un commit est une étape vérifiable, pas une sauvegarde automatique de chaque frappe. Faites un commit lorsque l'application atteint un état cohérent : le projet démarre, le contrôle Django passe et les fichiers nécessaires sont présents.

Checkpoint

À la fin de ce chapitre, les commandes suivantes doivent fonctionner :

```
uv run python manage.py check
uv run python manage.py runserver
uv run django-admin --version
git status
```

Exercices

1. Décrivez avec vos mots la différence entre un projet Django et une app Django.
2. Supprimez `.venv/` si elle existe, puis relancez une commande avec `uv run`. Que constatez-vous ?
3. Ajoutez une dépendance de développement avec `uv add --dev ruff`, puis observez la différence entre les dépendances normales et celles de développement dans `pyproject.toml`.
4. Faites un commit avant et après la création de l'app `cours`, puis utilisez `git diff` pour identifier ce que Django a généré.

Comprendre l'architecture Django et le routage

Le chemin complet d'une requête

Une bonne application Django n'est pas une collection de fichiers que l'on modifie au hasard. Elle est une chaîne de responsabilités. Pour une requête vers `/cours/12/`, le parcours conceptuel est :

```
navigateur
  requête HTTP

config/urls.py
  route choisie

vue Python
  lecture ou écriture

modèle / ORM    base de données

template    contexte Python
            HTML

navigateur
```

Django appelle souvent cette organisation **MVT** :

Élément Django	Responsabilité	Analogie MVC
Model	données, contraintes et accès à la base	Model
View	traitement de la requête et choix de la réponse	Controller
Template	représentation HTML	View

La comparaison avec MVC est utile mais imparfaite. Le nom importe moins que la règle : une vue ne devrait pas contenir tout le HTML, un template ne devrait pas interroger directement la base, et un modèle ne devrait pas connaître la mise en page. Le découpage rend chaque partie remplaçable et testable ([Django Software Foundation 2025c](#)). Les bonnes pratiques de conception Django insistent aussi sur la lisibilité, la simplicité et le respect des conventions du framework ([Greenfeld et Roy Greenfeld 2017](#)).

Les apps : découper par responsabilité

L'app `cours` aura les responsabilités liées au catalogue pédagogique. Dans une application qui grandit, on pourrait ajouter :

```
comptes/      utilisateurs, profils, permissions
cours/        matières et cours
inscriptions/ suivi de la progression
notifications/ courriels et alertes
```

Une app bien découpée peut être déplacée dans un autre projet avec peu de modifications. Il ne faut pas créer une app pour chaque modèle : on crée une app pour un domaine fonctionnel cohérent.

Lire `settings.py` plutôt que le copier

Le fichier `config/settings.py` configure le comportement du projet. Les sections importantes sont :

```
from pathlib import Path

BASE_DIR = Path(__file__).resolve().parent.parent

SECRET_KEY = "à-remplacer-en-production"
DEBUG = True
ALLOWED_HOSTS = []

INSTALLED_APPS = [
    # Apps Django fournies
    "django.contrib.admin",
    "django.contrib.auth",
    "django.contrib.contenttypes",
    "django.contrib.sessions",
    "django.contrib.messages",
    "django.contrib.staticfiles",
    # App du projet
    "cours",
]

MIDDLEWARE = [
    "django.middleware.security.SecurityMiddleware",
    "django.contrib.sessions.middleware.SessionMiddleware",
    "django.middleware.common.CommonMiddleware",
    "django.middleware.csrf.CsrfViewMiddleware",
    "django.contrib.auth.middleware.AuthenticationMiddleware",
    "django.contrib.messages.middleware.MessageMiddleware",
    "django.middleware.clickjacking.XFrameOptionsMiddleware",
]

ROOT_URLCONF = "config.urls"

TEMPLATES = [
    {
        "BACKEND": "django.template.backends.django.DjangoTemplates",
        "DIRS": [],
        "APP_DIRS": True,
        "OPTIONS": {
            "context_processors": [
                "django.template.context_processors.request",
                "django.contrib.auth.context_processors.auth",
                "django.contrib.messages.context_processors.messages",
            ],
        },
    },
]

WSGI_APPLICATION = "config.wsgi.application"
```

INSTALLED_APPS active les composants. MIDDLEWARE forme une pile traversée par les requêtes et réponses. TEMPLATES indique comment trouver et rendre les templates. ROOT_URLCONF indique quel routeur commencer à lire.

Le secret et le mode debug

SECRET_KEY est utilisé par plusieurs mécanismes cryptographiques de Django. Il ne doit pas être publié.

DEBUG = True affiche des informations très détaillées en cas d'erreur ; c'est utile localement et dangereux sur Internet. Les réglages de production seront traités au Section .

Déclarer les premières routes

Le routeur principal reçoit une liste de motifs. Modifiez `config/urls.py` :

```
from django.contrib import admin
from django.urls import include, path

urlpatterns = [
    path("admin/", admin.site.urls),
    path("cours/", include("cours.urls")),
]
```

Créez ensuite `cours/urls.py` :

```
from django.urls import path

from . import views

app_name = "cours"

urlpatterns = [
    path("", views.liste_cours, name="liste"),
]
```

Trois détails sont essentiels :

1. `include` délègue un préfixe à l'app ;
2. `app_name` crée un espace de noms ;
3. `name` donne un identifiant stable à la route.

Un template doit utiliser le nom de la route, pas recopier son URL. Ainsi, si `cours/` devient `catalogue/`, les liens générés par `{% url %}` restent cohérents après la modification du routeur.

Écrire une vue minimale

Dans `cours/views.py` :

```
from django.http import HttpRequest, HttpResponse

def accueil(request: HttpRequest) -> HttpResponse:
    return HttpResponse("Bienvenue dans BiblioCours")
```

Ajoutez temporairement la route :

```
urlpatterns = [
    path("", views.accueil, name="accueil"),
]
```

Ouvrez `/cours/`. Une vue reçoit au minimum un objet `HttpRequest` et renvoie un objet de réponse. `HttpResponse` est idéal pour comprendre le mécanisme, mais une application HTML utilisera plutôt `render`, présenté au chapitre Section .

Paramètres capturés dans l'URL

Django sait convertir une partie de l'URL avant d'appeler la vue :

```
urlpatterns = [
    path("<int:pk>/", views.detail_cours, name="detail"),
]
```

Pour `/cours/12/`, Django appelle :

```
def detail_cours(request: HttpRequest, pk: int) -> HttpResponse:
    return HttpResponse(f"Le cours demandé est {pk}")
```

Le convertisseur `int` refuse les valeurs qui ne sont pas des entiers. D'autres convertisseurs courants sont `str`, `slug`, `uuid` et `path`. Une URL porte donc une information structurée, et non une chaîne que la vue doit parser elle-même.

Comprendre la résolution des URLs

Django parcourt `urlpatterns` de haut en bas et s'arrête au premier motif correspondant. L'ordre compte :

```
urlpatterns = [
    path("nouveau/", views.nouveau_cours, name="nouveau"),
    path("<int:pk>/", views.detail_cours, name="detail"),
]
```

La route statique doit précéder la route dynamique. Sinon, selon les motifs, un chemin prévu pour `nouveau/` peut être interprété comme un identifiant.

Pour fabriquer une URL dans Python :

```
from django.urls import reverse

url = reverse("cours:detail", kwargs={"pk": 12})
# "/cours/12/"
```

Cette technique est préférable à `"/cours/12/"` en dur. Elle sert notamment dans les redirections et les tests.

Requêtes et réponses : les distinctions importantes

Une requête contient plusieurs catégories d'information :

```
def rechercher(request):
    terme = request.GET.get("q", "")
    langue = request.headers.get("Accept-Language")
    utilisateur = request.user
    return HttpResponse(f"Recherche: {terme}")
```

- `request.GET` contient les paramètres de l'URL, donc des données de lecture ;
- `request.POST` contient les données envoyées dans un formulaire ;
- `request.user` représente l'utilisateur courant, éventuellement anonyme ;
- les en-têtes décrivent le contexte HTTP, mais ne doivent pas être traités comme des données fiables ;
- `request.method` indique GET, POST, etc.

Une vue ne doit pas faire confiance aux données reçues. Même un champ qui semble numérique ou une valeur provenant d'un navigateur doit être validé côté serveur. Les formulaires Django aideront à centraliser cette validation.

Premier modèle mental à retenir

Quand une page ne fonctionne pas, diagnostiquez dans cet ordre :

1. l'URL est-elle réellement atteinte ?
2. la vue est-elle appelée avec les bons paramètres ?
3. la requête vers la base renvoie-t-elle le résultat attendu ?
4. le contexte transmis au template porte-t-il le bon nom ?
5. le template affiche-t-il ce nom et cette valeur ?
6. le navigateur reçoit-il le HTML attendu ?

Cette méthode est plus rapide que de modifier simultanément le modèle, la vue, le template et le CSS.

Exercice guidé

Ajoutez une page `/cours/a-propos/` :

1. écrivez une vue qui renvoie une `HttpResponse` ;
2. donnez-lui le nom `a_propos` ;
3. appelez-la depuis `cours/urls.py` ;
4. générez son adresse avec `reverse` dans le shell Django ;
5. ajoutez un test qui vérifie que la réponse a le statut HTTP 200.

💡 Question de conception

La route `/cours/12/` utilise un entier parce que la vue cherche un objet par clé primaire. Si l'URL devait être lisible, `/cours/python-pour-debutants/` pourrait utiliser un `slug`. Le choix de l'URL est à la fois une décision technique, une décision d'ergonomie et une décision de référencement.

Modéliser les données avec l'ORM et les migrations

Penser d'abord au domaine

Avant d'écrire `models.py`, décrivons les objets du métier. BiblioCours contient des **matières** et des **cours**. Une matière regroupe plusieurs cours ; un cours appartient à une seule matière. Cette phrase donne déjà une relation « un-à-plusieurs ».

Un bon modèle répond à des questions concrètes :

- quelles informations doivent être obligatoires ?
- quelle taille maximale une valeur peut-elle avoir ?
- deux enregistrements peuvent-ils porter la même valeur ?
- que se passe-t-il si la matière d'un cours est supprimée ?
- quelle date représente la création et quelle date représente la publication ?

Le modèle n'est pas seulement un miroir de la table SQL. Il formalise des règles et fournit une interface Python pour manipuler les données. L'ORM de Django traduit cette interface en requêtes SQL adaptées à la base configurée ([Django Software Foundation 2025e](#)).

Écrire les modèles de BiblioCours

Remplacez le contenu de `cours/models.py` par :

```
from django.db import models
from django.urls import reverse

class Matiere(models.Model):
    nom = models.CharField(max_length=100, unique=True)
    slug = models.SlugField(max_length=110, unique=True)
    description = models.TextField(blank=True)

    class Meta:
        ordering = ["nom"]
        verbose_name = "matière"
        verbose_name_plural = "matières"

    def __str__(self):
        return self.nom

class Cours(models.Model):
    class Statut(models.TextChoices):
        BROUILLON = "brouillon", "Brouillon"
        PUBLIE = "publie", "Publié"

    titre = models.CharField(max_length=200)
    slug = models.SlugField(max_length=220, unique=True)
```

```

resume = models.TextField(max_length=500)
contenu = models.TextField()
statut = models.CharField(
    max_length=20,
    choices=Statut.choices,
    default=Statut.BROUILLON,
)
matiere = models.ForeignKey(
    Matiere,
    on_delete=models.PROTECT,
    related_name="cours",
)
date_creation = models.DateTimeField(auto_now_add=True)
date_modification = models.DateTimeField(auto_now=True)
date_publication = models.DateTimeField(null=True, blank=True)

class Meta:
    ordering = ["-date_creation"]
    indexes = [
        models.Index(fields=["statut", "-date_creation"]),
    ]

def __str__(self):
    return self.titre

def get_absolute_url(self):
    return reverse("cours:detail", kwargs={"slug": self.slug})

@property
def est_publie(self):
    return self.statut == self.Statut.PUBLIE

```

Pourquoi ces champs ?

`CharField` représente une chaîne bornée. `max_length` est utile pour la base, les formulaires et la documentation de l'intention. `TextField` convient au contenu dont la taille est plus libre. `SlugField` représente une valeur adaptée aux URLs, généralement composée de lettres minuscules, chiffres et tirets.

`unique=True` crée une contrainte d'unicité. Ce n'est pas seulement une validation d'interface : la base elle-même empêchera deux matières d'avoir le même slug. La base est la dernière barrière, car deux requêtes concurrentes peuvent passer simultanément une validation purement applicative.

`TextChoices` évite les chaînes dispersées dans le code. La valeur enregistrée est "publie" et le libellé affiché est "Publié". On peut écrire `Cours.Statut.PUBLIE` dans le code et laisser Django gérer les choix.

`auto_now_add` est évalué à la création ; `auto_now` à chaque sauvegarde. `null=True` concerne la valeur en base, tandis que `blank=True` concerne la validation des formulaires. Cette différence est importante : `blank=True` ne rend pas automatiquement une colonne SQL nullable.

Choisir `on_delete`

La relation vers `Matiere` utilise `PROTECT`. Supprimer une matière qui possède encore des cours serait probablement une erreur métier ; Django lève donc une exception plutôt que d'effacer silencieusement le catalogue.

Les comportements courants sont :

- `CASCADE` : supprimer les objets liés ;

- PROTECT : refuser la suppression ;
- SET_NULL : mettre la relation à NULL si `null=True` ;
- SET_DEFAULT : utiliser la valeur par défaut ;
- DO_NOTHING : ne rien orchestrer, choix réservé aux cas maîtrisés.

Il n'existe pas de choix universel. Pour une photo liée à un profil, **CASCADE** peut être logique. Pour une facture liée à un client, **PROTECT** est souvent plus sûr.

Les migrations : l'historique du schéma

Un modèle Python et une table SQL ne sont pas la même chose. La migration est le fichier qui décrit une transformation du schéma. Après la modification des modèles :

```
uv run python manage.py makemigrations cours
uv run python manage.py migrate
```

`makemigrations` compare l'état actuel des modèles avec l'historique des migrations et génère un fichier Python. `migrate` exécute les migrations non appliquées dans la base.

Inspectez ce que Django prévoit :

```
uv run python manage.py showmigrations
uv run python manage.py sqlmigrate cours 0001
```

`sqlmigrate` montre le SQL généré sans l'exécuter. C'est un excellent outil pour relier le code Python à la réalité de la base.

 Ne modifiez pas une migration déjà partagée

Une migration appliquée fait partie de l'histoire du schéma. En équipe, créez une nouvelle migration plutôt que de réécrire une ancienne. Modifier une migration déjà exécutée peut créer des divergences entre les bases des collaborateurs et la base de production.

Les fichiers de migration doivent être versionnés. La base SQLite locale ne l'est généralement pas, mais les instructions permettant de la recréer le sont.

Créer et lire des objets dans le shell

```
uv run python manage.py shell
```

```
from cours.models import Cours, Matiere

informatique = Matiere.objects.create(
    nom="Informatique",
    slug="informatique",
    description="Comprendre les outils numériques.",
)

python_cours = Cours.objects.create(
    titre="Python pour débutants",
    slug="python-pour-debutants",
    resume="Les bases du langage Python.",
    contenu="Variables, fonctions et modules.",
)
```

```

    matiere=informatique,
    statut=Cours.Statut.PUBLIE,
)

python_cours.matiere
informatique.cours.all()
Cours.objects.filter(statut=Cours.Statut.PUBLIE)

```

Chaque accès à `.objects` construit ou exécute un `QuerySet`. Un `QuerySet` est évaluable de manière différée : écrire `cours = Cours.objects.filter(...)` ne signifie pas nécessairement qu'une requête SQL est déjà partie. L'évaluation arrive lors d'une itération, d'une conversion en liste, d'un `len`, ou de l'accès à certains résultats.

Filtrer, trier et limiter

```

Cours.objects.filter(titre__icontains="python")
Cours.objects.filter(matiere__slug="informatique")
Cours.objects.filter(
    statut=Cours.Statut.PUBLIE,
    matiere__nom__iexact="informatique",
)
Cours.objects.exclude(statut=Cours.Statut.BROUILLON)
Cours.objects.order_by("-date_publication")[:10]

```

Les doubles underscores traversent des relations et expriment des recherches. `filter` renvoie un `QuerySet`, `get` attend exactement un résultat et lève une exception si aucun ou plusieurs objets sont trouvés :

```
cours = Cours.objects.get(slug="python-pour-debutants")
```

Pour une vue, `get_object_or_404` transforme l'absence en réponse HTTP 404, plutôt que de laisser remonter une exception non gérée.

select_related et prefetch_related

Afficher une liste de cours et le nom de leur matière peut provoquer une requête par cours si l'on ne prépare pas la relation. C'est le problème classique du N+1 :

```

cours = Cours.objects.select_related("matiere").filter(
    statut=Cours.Statut.PUBLIE,
)

```

`select_related` utilise une jointure SQL pour les relations à valeur unique comme `ForeignKey`. Pour une relation inverse ou plusieurs-à-plusieurs, utilisez `prefetch_related` :

```
matieres = Matiere.objects.prefetch_related("cours").all()
```

Il ne faut pas ajouter ces méthodes mécaniquement. Mesurez les requêtes et choisissez l'outil adapté à la relation et à l'écran.

Modifier les données avec prudence

Ces deux formes ne sont pas équivalentes :

```
cours = Cours.objects.get(pk=1)
cours.titre = "Nouveau titre"
cours.save(update_fields=["titre", "date_modification"])
```

et :

```
Cours.objects.filter(pk=1).update(titre="Nouveau titre")
```

`save` passe par le cycle de sauvegarde de l'instance. `update` exécute une mise à jour SQL directe et ne déclenche pas les méthodes `save` personnalisées. Pour une opération en masse, `update` est efficace, mais il faut connaître ses conséquences.

Pour une opération composée de plusieurs écritures, utilisez une transaction :

```
from django.db import transaction

@transaction.atomic
def publier(cours):
    cours.statut = Cours.Statut.PUBLIE
    cours.save(update_fields=["statut", "date_modification"])
```

Si une exception survient dans le bloc atomique, Django annule les opérations de la transaction. La transaction protège la cohérence des données ; elle ne remplace pas la validation métier.

Concevoir avant de migrer

Ajoutez les champs uniquement lorsqu'ils correspondent à un besoin. Un champ `description` et un champ `contenu` ne sont pas interchangeables si leurs contraintes et leurs usages diffèrent. À l'inverse, ne créez pas un champ `est_publie` en plus de `statut` : deux sources de vérité peuvent diverger.

Le bon flux est :

1. décrire la règle métier ;
2. choisir le type et les contraintes ;
3. modifier le modèle ;
4. lire la migration générée ;
5. appliquer et tester la migration ;
6. essayer les cas normaux et les cas limites.

Exercices

1. Ajoutez un champ `niveau` avec trois choix : débutant, intermédiaire, avancé.
2. Empêchez la création d'un cours publié sans date de publication en ajoutant une validation de modèle adaptée.
3. Écrivez une requête qui retourne les cinq cours publiés les plus récents d'une matière donnée.
4. Activez les logs SQL en développement et comparez une liste avec et sans `select_related("matiere")`.

Construire les pages : vues, URLs et templates

De la donnée à une page HTML

Une page dynamique ne devrait pas construire son HTML par concaténation de chaînes Python. La vue prépare les données et choisit le template ; le template décrit la représentation HTML. Cette séparation rend le HTML lisible par une personne qui travaille sur l'interface sans lui demander de modifier la logique métier.

Pour BiblioCours, nous voulons :

/cours/	liste des cours publiés
/cours/<slug>/	détail d'un cours
/cours/matiere/<slug>/	cours d'une matière

Les noms de routes restent stables même si leur forme évolue. C'est pourquoi nous avons défini `app_name = "cours"` et des noms explicites ([Django Software Foundation 2025h](#)).

Les vues de liste et de détail

Complétez `cours/urls.py` :

```
from django.urls import path

from . import views

app_name = "cours"

urlpatterns = [
    path("matiere/<slug:slug>/", views.cours_par_matiere, name="par_matiere"),
    path("", views.liste_cours, name="liste"),
    path("<slug:slug>/", views.detail_cours, name="detail"),
]
```

L'ordre mérite attention : la route `matiere/<slug:slug>/` doit apparaître avant `"<slug:slug>/"`, sinon le mot `matiere` peut être capturé comme le slug d'un cours.

Dans `cours/views.py` :

```
from django.core.paginator import Paginator
from django.db.models import Q
from django.shortcuts import get_object_or_404, render

from .models import Cours, Matiere

def liste_cours(request):
    """Affiche les cours publiés, dix par page."""
    queryset = (
```

```
Cours.objects
    .filter(statut=Cours.Statut.PUBLIE)
    .select_related("matiere")
)
paginator = Paginator(queryset, 10)
page_number = request.GET.get("page")
page_obj = paginator.get_page(page_number)

return render(
    request,
    "cours/liste.html",
    {"page_obj": page_obj},
)

def detail_cours(request, slug):
    cours = get_object_or_404(
        Cours.objects.select_related("matiere"),
        slug=slug,
        statut=Cours.Statut.PUBLIE,
    )
    return render(request, "cours/detail.html", {"cours": cours})

def cours_par_matiere(request, slug):
    matiere = get_object_or_404(Matiere, slug=slug)
    cours = (
        matiere.cours
        .filter(statut=Cours.Statut.PUBLIE)
        .select_related("matiere")
    )
    return render(
        request,
        "cours/liste.html",
        {"page_obj": cours, "matiere": matiere},
    )
```

Pourquoi render ?

`render(request, template_name, context) :`

1. charge le template ;
2. lui transmet un dictionnaire de contexte ;
3. produit une réponse HTTP avec le HTML.

La clé "page_obj" est un contrat entre la vue et le template. Si la vue emploie "cours" mais que le template parcourt page_obj, la page semble vide sans que la base soit nécessairement en cause.

404 plutôt que page vide

`get_object_or_404` exprime qu'un objet demandé doit exister. Une URL inconnue renvoie alors un statut 404 Not Found, ce qui est correct pour le navigateur, les moteurs de recherche et les tests. Il ne faut pas attraper toutes les exceptions et renvoyer une page générique : cela masque les erreurs de programmation et transforme parfois un 404 en 500.

Organiser les templates

Créez :

```
cours/
  templates/
    cours/
      base.html
      liste.html
      detail.html
      partials/
        carte_cours.html
```

Avec `APP_DIRS = True`, Django cherche un dossier `templates` dans les apps installées. Le sous-dossier `cours/` évite les collisions si deux apps possèdent un fichier `liste.html`.

Le template parent

`cours/templates/cours/base.html` :

```
{% load static %}
<!doctype html>
<html lang="fr">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>{% block title %}BiblioCours{% endblock %}</title>
  <link rel="stylesheet" href="{% static 'cours/site.css' %}">
</head>
<body>
  <header class="site-header">
    <nav aria-label="Navigation principale">
      <a href="{% url 'cours:liste' %}">BiblioCours</a>
      <a href="{% url 'cours:liste' %}">Tous les cours</a>
    </nav>
  </header>

  <main class="container">
    {% block content %}{% endblock %}
  </main>

  <footer class="site-footer">
    <p>BiblioCours - apprendre en construisant.</p>
  </footer>
</body>
</html>
```

`{% extends %}` et les blocs permettent aux pages enfants de réutiliser le document entier. `block title` et `block content` sont des points d'extension. Cette technique applique le principe DRY sans rendre toutes les pages identiques.

Afficher une collection avec `liste.html`

```
{% extends "cours/base.html" %}

{% block title %}
  {% if matiere %}Cours de {{ matiere.nom }}{% else %}Tous les cours{% endif %}
  - BiblioCours
{% endblock %}

{% block content %}
  {% if matiere %}
    <header class="page-heading">
      <p class="eyebrow">Matière</p>
      <h1>{{ matiere.nom }}</h1>
      <p>{{ matiere.description }}</p>
    </header>
  {% else %}
    <header class="page-heading">
      <p class="eyebrow">Catalogue</p>
      <h1>Tous les cours</h1>
    </header>
  {% endif %}

  <section aria-labelledby="cours-heading">
    <h2 id="cours-heading">Cours disponibles</h2>
    {% for cours in page_obj %}
      {% include "cours/partials/carte_cours.html" %}
    {% empty %}
      <p>Aucun cours publié pour le moment.</p>
    {% endfor %}
  </section>

  {% if page_obj.has_other_pages %}
    <nav aria-label="Pagination" class="pagination">
      {% if page_obj.has_previous %}
        <a href="?page={{ page_obj.previous_page_number }}">Précédent</a>
      {% endif %}
      <span>Page {{ page_obj.number }} sur {{ page_obj.paginator.num_pages }}</span>
      {% if page_obj.has_next %}
        <a href="?page={{ page_obj.next_page_number }}">Suivant</a>
      {% endif %}
    </nav>
  {% endif %}
{% endblock %}
```

Le bloc `{% empty %}` évite une page silencieuse quand le QuerySet ne contient aucun élément. Les attributs `has_previous` et `has_next` appartiennent à l'objet `Page` produit par `Paginator`.

Le partial `partials/carte_cours.html` :

```
<article class="course-card">
  <p class="eyebrow">{{ cours.matiere.nom }}</p>
  <h3>
    <a href="{{ cours.get_absolute_url }}">{{ cours.titre }}</a>
  </h3>
  <p>{{ cours.resume }}</p>
  <p class="meta">
    Publié le {{ cours.date_publication|date:"d/m/Y" }}
  </p>
</article>
```

`include` rend un fragment réutilisable dans le contexte courant. Le modèle fournit `get_absolute_url`, ce qui centralise la construction du lien.

Afficher le détail

`cours/templates/cours/detail.html` :

```
{% extends "cours/base.html" %}

{% block title %}{{ cours.titre }} - BiblioCours{% endblock %}

{% block content %}
<article class="course-detail">
  <p class="eyebrow">
    <a href="{% url 'cours:par_matiere' cours.matiere.slug %}">
      {{ cours.matiere.nom }}
    </a>
  </p>
  <h1>{{ cours.titre }}</h1>
  <p class="meta">
    Mis à jour le {{ cours.date_modification|date:"d/m/Y à H:i" }}
  </p>
  <p class="lead">{{ cours.resume }}</p>
  <div class="course-content">
    {{ cours.contenu|linebreaks }}
  </div>
</article>
{% endblock %}
```

Les variables sont échappées automatiquement. C'est un mécanisme essentiel : si le contenu contient `<script>`, Django l'affiche comme du texte au lieu de l'exécuter. Le filtre `linebreaks` transforme des retours à la ligne en paragraphes et éléments `<br>`.

N'utilisez pas `|safe` simplement pour « faire marcher » un affichage. Il désactive l'échappement et doit être réservé à du HTML produit par une source de confiance, nettoyé avec une stratégie explicite. La sécurité des templates est expliquée au Section .

Rechercher avec un paramètre GET

Ajoutons une recherche simple. Dans la vue :

```
def liste_cours(request):
    terme = request.GET.get("q", "").strip()
    queryset = (
        Cours.objects
        .filter(statut=Cours.Statut.PUBLIE)
        .select_related("matiere")
    )
    if terme:
        queryset = queryset.filter(
            Q(titre__icontains=terme)
            | Q(resume__icontains=terme)
        )
    paginator = Paginator(queryset, 10)
    return render(
```

```
request,
"cours/liste.html",
{"page_obj": paginator.get_page(request.GET.get("page")), "terme": terme},
)
```

Les paramètres GET sont des chaînes fournies par l'utilisateur ; `icontains` protège contre l'injection SQL parce que l'ORM paramètre la requête, mais il faut tout de même limiter les coûts et la longueur dans une application publique.

Le formulaire de recherche peut être placé dans le template :

```
<form method="get" role="search">
  <label for="q">Rechercher un cours</label>
  <input id="q" name="q" value="{{ terme }}" type="search">
  <button type="submit">Rechercher</button>
</form>
```

Lire un bug de template

Pour une page vide, ne commencez pas par changer le CSS. Vérifiez :

```
print(queryset.query)
print(queryset.count())
```

Puis inspectez le contrat :

- le contexte contient-il `page_obj` ?
- la boucle utilise-t-elle `page_obj` ?
- le filtre `statut` exclut-il vos données de test ?
- la route pointe-t-elle vers la bonne vue ?
- le fichier est-il bien sous `templates/cours/` ?

Le mode DEBUG fournit aussi des pages d'erreur détaillées, mais il ne faut jamais en laisser l'affichage actif en production.

Exercices

1. Ajoutez un lien « Voir la matière » sur chaque carte.
2. Conservez le terme `q` dans les liens de pagination.
3. Créez une page 404 personnalisée avec un template `404.html`.
4. Écrivez une vue qui affiche uniquement les cours d'une matière et vérifiez qu'une matière inexistante renvoie bien 404.

Saisir et administrer les données : formulaires et admin

Pourquoi un formulaire HTML ne suffit pas

Un formulaire est une frontière entre le navigateur et l'application. Le navigateur peut aider avec `required`, `type="email"` ou `maxlength`, mais ces attributs ne sont pas des protections : n'importe quel client HTTP peut les contourner. La validation importante doit être exécutée côté serveur.

Django distingue trois étapes :

1. **liaison** : associer les données reçues à un formulaire ;
2. **validation** : vérifier les champs et les règles métier ;
3. **action** : sauvegarder ou effectuer une opération seulement si le formulaire est valide.

Un formulaire non valide doit être réaffiché avec les valeurs acceptables et les erreurs lisibles. Un formulaire valide doit éviter de créer deux fois la même ressource lorsque l'utilisateur actualise la page.

Ce dernier problème est résolu par le schéma **POST/Redirect/GET**.

Créer un ModelForm

Créez `cours/forms.py` :

```
from django import forms

from .models import Cours

class CoursForm(forms.ModelForm):
    class Meta:
        model = Cours
        fields = ["titre", "slug", "resume", "contenu", "matiere"]
        widgets = {
            "resume": forms.Textarea(attrs={"rows": 3}),
            "contenu": forms.Textarea(attrs={"rows": 12}),
        }

    def clean_titre(self):
        titre = self.cleaned_data["titre"].strip()
        if len(titre) < 5:
            raise forms.ValidationError(
                "Le titre doit comporter au moins cinq caractères."
            )
        return titre

    def clean(self):
        cleaned_data = super().clean()
        titre = cleaned_data.get("titre")
        resume = cleaned_data.get("resume")
```

```
if titre and resume and titre.lower() in resume.lower():
    self.add_error(
        "resume",
        "Le résumé ne doit pas répéter le titre mot pour mot.",
    )
return cleaned_data
```

ModelForm déduit les champs à partir du modèle et réutilise ses contraintes de base. La liste `fields` doit être explicite : `fields = "__all__"` risque d'exposer un champ ajouté plus tard sans que l'on s'en rende compte ([Django Software Foundation 2025i](#)).

Les trois niveaux de validation

- `clean_titre` valide un champ et se concentre sur sa valeur ;
- `clean` valide une relation entre plusieurs champs ;
- `Model.clean` peut porter une règle de modèle indépendante de ce formulaire.

Une validation ne doit pas seulement servir à afficher un message. Elle doit protéger l'invariant métier. Par exemple, si un cours publié doit toujours avoir un résumé, cette règle devrait être appliquée dans tous les chemins de création, pas uniquement sur la page HTML.

La vue de création

Ajoutez une route protégée plus tard par authentification :

```
path("nouveau/", views.creer_cours, name="creer"),
```

Puis la vue :

```
from django.contrib import messages
from django.contrib.auth.decorators import login_required
from django.shortcuts import get_object_or_404, redirect, render

from .forms import CoursForm

@login_required
def creer_cours(request):
    if request.method == "POST":
        form = CoursForm(request.POST)
        if form.is_valid():
            cours = form.save()
            messages.success(request, f"Le cours « {cours.titre} » a été créé.")
            return redirect(cours.get_absolute_url())
    else:
        form = CoursForm()

    return render(request, "cours/formulaire.html", {"form": form})
```

La méthode HTTP est une décision de sécurité et de sémantique :

- GET affiche le formulaire et ne doit pas modifier l'état ;
- POST transmet une intention de modification ;
- une redirection après succès empêche un rafraîchissement de répéter le POST.

Le décorateur `login_required` redirige les utilisateurs anonymes vers la page de connexion. Nous verrons la configuration de cette redirection au chapitre Section .

Le template du formulaire

cours/templates/cours/formulaire.html :

```
{% extends "cours/base.html" %}

{% block title %}Nouveau cours - BiblioCours{% endblock %}

{% block content %}
<h1>Créer un cours</h1>

<form method="post" novalidate>
  {% csrf_token %}

  {% if form.non_field_errors %}
    <div class="errors" role="alert">
      {{ form.non_field_errors }}
    </div>
  {% endif %}

  {% for field in form %}
    <div class="form-field">
      {{ field.label_tag }}
      {{ field }}
      {% if field.help_text %}
        <p class="help">{{ field.help_text }}</p>
      {% endif %}
      {% for error in field.errors %}
        <p class="error" role="alert">{{ error }}</p>
      {% endfor %}
    </div>
  {% endfor %}

  <button type="submit">Enregistrer</button>
  <a href="{% url 'cours:liste' %}">Annuler</a>
</form>
{% endblock %}
```

`{% csrf_token %}` est obligatoire pour un POST provenant d'un formulaire Django. Le jeton associe la requête à la session et protège contre une partie des attaques CSRF. Il ne remplace pas l'authentification ni l'autorisation.

`novalidate` désactive la validation native du navigateur afin de rendre visibles les messages Django pendant l'apprentissage. Dans un projet réel, vous pouvez conserver la validation native en complément ; elle améliore l'expérience mais ne remplace jamais la validation serveur.

Modifier un objet existant

Le même `ModelForm` peut être lié à une instance :

```
@login_required
def modifier_cours(request, slug):
    cours = get_object_or_404(Cours, slug=slug)

    if request.method == "POST":
        form = CoursForm(request.POST, instance=cours)
```

```
    if form.is_valid():
        form.save()
        messages.success(request, "Le cours a été mis à jour.")
        return redirect(cours.get_absolute_url())
    else:
        form = CoursForm(instance=cours)

    return render(
        request,
        "cours/formulaire.html",
        {"form": form, "cours": cours},
    )
```

La présence d'`instance` change le comportement : sans elle, `save()` crée un objet ; avec elle, `save()` met à jour l'objet. La route de modification doit être distincte de la route de création et doit vérifier que l'utilisateur a le droit de modifier ce cours.

Supprimer : préférer une confirmation explicite

La suppression ne devrait pas être déclenchée par un lien GET :

```
<!-- À éviter : un robot ou un clic involontaire pourrait supprimer -->
<a href="{% url 'cours:supprimer' cours.slug %}">Supprimer</a>
```

Utilisez un formulaire POST avec CSRF et demandez une confirmation dans une page dédiée :

```
@login_required
def supprimer_cours(request, slug):
    cours = get_object_or_404(Cours, slug=slug)
    if request.method == "POST":
        titre = cours.titre
        cours.delete()
        messages.success(request, f"Le cours « {titre} » a été supprimé.")
        return redirect("cours:liste")
    return render(request, "cours/confirmer_suppression.html", {"cours": cours})
```

L'approbation visuelle n'est pas une mesure de sécurité ; l'autorisation et le POST protégé le sont. Une opération irréversible doit aussi être reflétée dans les tests et, selon le domaine, dans une stratégie d'archivage plutôt que de suppression physique.

L'administration Django

L'admin est une interface interne puissante pour les personnes qui gèrent le contenu. Elle ne remplace pas une interface publique : ses écrans, ses permissions et son modèle mental sont ceux d'un back-office. Elle permet néanmoins de peupler BiblioCours rapidement.

Dans `cours/admin.py` :

```
from django.contrib import admin

from .models import Cours, Matiere
```

```
@admin.register(Matiere)
class MatiereAdmin(admin.ModelAdmin):
    list_display = ["nom", "slug"]
    search_fields = ["nom", "description"]
    prepopulated_fields = {"slug": ("nom",)}

@admin.register(Cours)
class CoursAdmin(admin.ModelAdmin):
    list_display = [
        "titre",
        "matiere",
        "statut",
        "date_creation",
        "date_publication",
    ]
    list_filter = ["statut", "matiere", "date_creation"]
    search_fields = ["titre", "resume", "contenu"]
    autocomplete_fields = ["matiere"]
    prepopulated_fields = {"slug": ("titre",)}
    readonly_fields = ["date_creation", "date_modification"]
```

Créez un compte administrateur :

```
uv run python manage.py createsuperuser
uv run python manage.py runserver
```

Visitez <http://127.0.0.1:8000/admin/>. `list_display` rend la liste utile, `list_filter` ajoute des filtres latéraux et `search_fields` construit une recherche. `readonly_fields` évite qu'une date technique soit modifiée à la main.

L'admin et les performances

Si une liste affiche une relation, Django peut charger chaque matière séparément. Ajoutez une optimisation à l'admin :

```
class CoursAdmin(admin.ModelAdmin):
    # autres options...

    def get_queryset(self, request):
        return super().get_queryset(request).select_related("matiere")
```

Mesurez avant d'optimiser, mais ne rendez pas l'admin inutilisable avec une requête N+1 lorsque le catalogue grandit.

Messages utilisateur

Les messages sont temporaires et associés à la prochaine réponse :

```
from django.contrib import messages

messages.success(request, "Le cours a été enregistré.")
messages.error(request, "Impossible de supprimer ce cours.")
```

Dans `base.html` :

```
{% if messages %}
<div class="messages" aria-live="polite">
  {% for message in messages %}
    <p class="message message-{{ message.tags }}">{{ message }}</p>
  {% endfor %}
</div>
{% endif %}
```

Le contexte est disponible grâce à `django.contrib.messages.context_processors.messages`, présent dans la configuration initiale. Un message ne doit pas être la seule façon de communiquer une erreur : le formulaire doit aussi porter les erreurs au niveau du champ.

Exercices

1. Ajoutez un `ModelForm` pour `Matiere` avec une validation de nom unique insensible à la casse.
2. Ajoutez une action d'administration qui publie les cours sélectionnés.
3. Faites en sorte que seuls les utilisateurs ayant une permission puissent créer un cours.
4. Testez les trois scénarios de `creer_cours` : GET, POST valide et POST invalide.

Authentification, sécurité et fichiers statiques

Authentifier n'est pas autoriser

Deux questions différentes sont souvent confondues :

- **authentification** : « qui est cette personne ? » ;
- **autorisation** : « a-t-elle le droit de faire cette action ? ».

Un utilisateur connecté peut avoir le droit de lire un cours mais pas de le supprimer. La présence de `request.user` ne suffit donc jamais à autoriser une opération.

Django fournit un système d'utilisateurs, de sessions, de groupes et de permissions. Il faut le configurer et l'utiliser de manière explicite plutôt que d'inventer un mécanisme de mot de passe. Les recommandations de sécurité du framework et les contrôles OWASP complètent cette compréhension ([Django Software Foundation 2025f](#); [OWASP Foundation 2021](#)).

Ajouter les routes d'authentification

Dans `config/urls.py` :

```
from django.contrib import admin
from django.contrib.auth import views as auth_views
from django.urls import include, path

urlpatterns = [
    path("admin/", admin.site.urls),
    path("cours/", include("cours.urls")),
    path(
        "connexion/",
        auth_views.LoginView.as_view(template_name="registration/login.html"),
        name="login",
    ),
    path("deconnexion/", auth_views.LogoutView.as_view(), name="logout"),
]
```

Ajoutez dans `config/settings.py` :

```
LOGIN_URL = "login"
LOGIN_REDIRECT_URL = "cours:liste"
LOGOUT_REDIRECT_URL = "cours:liste"
```

Le template de connexion doit se trouver à l'emplacement attendu :

```
templates/
  registration/
    login.html
```

Pour que ce dossier global soit trouvé, ajoutez-le à `TEMPLATES[0][\"DIRS\"]` :

```
TEMPLATES[0][ "DIRS" ] = [BASE_DIR / "templates"]
```

templates/registration/login.html :

```
{% extends "cours/base.html" %}

{% block title %}Connexion - BiblioCours{% endblock %}

{% block content %}
<h1>Se connecter</h1>
<form method="post">
    {% csrf_token %}
    {{ form.as_p }}
    <button type="submit">Se connecter</button>
</form>
{% endblock %}
```

Dans base.html, adaptez la navigation :

```
{% if user.is_authenticated %}
<span>Bonjour {{ user.get_username }}</span>
<form method="post" action="{% url 'logout' %}">
    {% csrf_token %}
    <button type="submit">Se déconnecter</button>
</form>
{% else %}
<a href="{% url 'login' %}?next={{ request.path }}">
    Se connecter
</a>
{% endif %}
```

Sur les versions récentes de Django, la déconnexion est traitée par POST. Utiliser un formulaire respecte le fait que la déconnexion modifie l'état de la session.

Protéger une vue et une action

Le décorateur protège la présence d'une session :

```
from django.contrib.auth.decorators import login_required

@login_required
def creer_cours(request):
    ...
```

Pour une permission :

```
from django.contrib.auth.decorators import permission_required

@permission_required("cours.add_cours", raise_exception=True)
def creer_cours(request):
    ...
```

`raise_exception=True` renvoie une réponse interdite (403) au lieu de rediriger silencieusement. Choisissez le comportement en fonction du contexte. Une app publique peut vouloir masquer l'existence d'une ressource ; une interface interne doit souvent expliquer le manque de permission.

Dans l'admin, les permissions `add`, `change`, `delete` et `view` sont associées aux modèles. On peut affecter ces permissions à un groupe « Éditeurs » sans donner un accès complet de superutilisateur.

CSRF : protéger les requêtes qui changent l'état

Une attaque CSRF amène le navigateur d'une victime déjà connectée à envoyer une requête vers un site où elle est authentifiée. Le navigateur peut joindre les cookies automatiquement ; le serveur doit donc exiger un jeton que le site attaquant ne peut pas lire.

Dans un formulaire interne :

```
<form method="post">
  {% csrf_token %}
  ...
</form>
```

Pour un appel JavaScript, le jeton doit être envoyé selon la configuration Django. Ne désactivez pas `CsrfViewMiddleware` pour contourner une erreur : cherchez plutôt pourquoi le cookie ou le jeton ne correspond pas ([Django Software Foundation 2025a](#)).

Échappement, injection et entrées utilisateur

XSS et templates

Django échappe les variables dans les templates par défaut :

```
{{ commentaire.texte }}
```

Si un utilisateur enregistre `<img src=x onerror=alert(1)>`, cette valeur doit rester du texte. Le filtre `safe` supprime la protection :

```
{{ contenu_html|safe }}
```

Ne l'utilisez que si le contenu a été nettoyé par une bibliothèque adaptée à votre format et à votre modèle de menace. Le fait qu'une donnée vienne de l'admin ne garantit pas que tous les administrateurs soient dignes de confiance ni que le champ soit toujours contrôlé.

Injection SQL

L'ORM paramètre les valeurs :

```
Cours.objects.filter(titre__icontains=terme)
```

Évitez de construire du SQL avec une f-string. Si une requête SQL brute est nécessaire, utilisez des paramètres et expliquez la raison dans le code :

```
Cours.objects.raw(
    "SELECT * FROM cours_cours WHERE titre LIKE %s",
    [f"%{terme}%"],
)
```

Le SQL brut n'est pas « plus professionnel ». Il augmente la responsabilité du développeur concernant les paramètres, les noms de tables, les performances et la portabilité.

Redirections

Ne redirigez pas aveuglément vers une URL fournie par l'utilisateur. Un paramètre `next` doit être validé comme une destination sûre. Les utilitaires de Django et la configuration des hôtes autorisés aident à limiter les redirections ouvertes.

Secrets et configuration

En développement, `SECRET_KEY` peut être remplacé par une valeur locale. En production, lisez-la depuis une variable d'environnement :

```
import os

SECRET_KEY = os.environ["DJANGO_SECRET_KEY"]
DEBUG = os.environ.get("DJANGO_DEBUG", "false").lower() == "true"
ALLOWED_HOSTS = [
    host.strip()
    for host in os.environ.get("DJANGO_ALLOWED_HOSTS", "").split(",")
    if host.strip()
]
```

Ne committez ni clé secrète, ni mot de passe de base, ni token de service. Un fichier `.env` peut aider en local mais doit être ignoré par Git et ne doit pas être copié dans une archive de déploiement. En production, utilisez le système de secrets de l'hébergeur.

Fichiers statiques

Les fichiers statiques sont le CSS, le JavaScript et les images livrés avec le code. Configurez `config/settings.py` :

```
STATIC_URL = "static/"
STATICFILES_DIRS = [BASE_DIR / "static"]
STATIC_ROOT = BASE_DIR / "staticfiles"
```

Ajoutez :

```
static/
  cours/
    site.css
```

Le template les charge avec `{% load static %}` et `{% static 'cours/site.css' %}`. En développement, `runserver` les sert si `DEBUG=True`. En production, la commande suivante rassemble les fichiers :

```
uv run python manage.py collectstatic
```

STATIC_ROOT est un dossier de sortie ; il ne doit pas être confondu avec STATICFILES_DIRS, qui contient les sources. Un serveur web ou un stockage objet servira ensuite le contenu rassemblé.

Téléversements et fichiers média

Un fichier envoyé par un utilisateur n'est pas un fichier statique. Configurez :

```
MEDIA_URL = "media/"
MEDIA_ROOT = BASE_DIR / "media"
```

Dans un modèle :

```
document = models.FileField(upload_to="cours/%Y/%m", blank=True)
```

Un upload doit être limité par taille et type, renommé par Django et stocké avec des permissions correctes. Ne servez pas aveuglément un fichier comme du HTML actif. Pour des données sensibles ou volumineuses, un stockage séparé avec des URLs temporaires est souvent plus adapté.

Checklist de sécurité locale

Avant de publier :

- DEBUG=False ;
- SECRET_KEY fourni par l'environnement ;
- ALLOWED_HOSTS renseigné ;
- HTTPS activé et cookies sécurisés configurés ;
- protection CSRF conservée ;
- permissions testées avec un utilisateur normal ;
- pas de secret dans Git ;
- uploads limités et traités comme non fiables ;
- python manage.py check --deploy exécuté.

La sécurité n'est pas une étape à la fin. Chaque champ, route et action doit répondre à la question : **qui peut faire quoi, avec quelles données, et que se passe-t-il si la requête est fabriquée à la main ?**

Exercices

1. Créez un groupe **Éditeurs** et donnez-lui uniquement les permissions utiles pour créer et modifier des cours.
2. Ajoutez une page qui n'est visible que par les utilisateurs connectés.
3. Testez un POST sans CSRF et observez la réponse ; ne désactivez pas la protection.
4. Configurez un fichier CSS et vérifiez qu'il est trouvé en développement puis rassemblé par collectstatic.

Tester, diagnostiquer et faire évoluer le code

Un test décrit un contrat

Un test utile ne vérifie pas que l'implémentation ressemble à ce que le développeur avait en tête. Il vérifie un comportement observable :

lorsqu'un visiteur demande la liste, la réponse est 200 et ne montre que les cours publiés.

Cette formulation résiste mieux aux refactorings que « la vue appelle telle fonction ». Les tests de Django peuvent utiliser une base de données de test, un client HTTP de test et les outils `reverse` et `assertContains` ([Django Software Foundation 2025g](#)). Cette manière de partir du comportement attendu rejoint l'approche de développement piloté par les tests décrite par Percival et Gregory ([Percival et Gregory 2024](#)).

Lancer la suite

Django fournit une commande intégrée :

```
uv run python manage.py test
```

Elle découvre les classes `SimpleTestCase`, `TestCase` et les fonctions de test dans les modules dont le nom commence généralement par `test`. La base de test est créée temporairement et détruite à la fin.

Ajoutez un dossier dédié :

```
cours/  
  tests/  
    __init__.py  
    test_models.py  
    test_views.py  
    test_forms.py
```

Tester le modèle

`cours/tests/test_models.py` :

```
from django.test import TestCase  
  
from cours.models import Cours, Matiere  
  
class CoursModelTests(TestCase):  
    @classmethod  
    def setUpTestData(cls):  
        cls.matiere = Matiere.objects.create(  
            nom="Informatique",
```

```

        slug="informatique",
    )

    def test_str_retourne_le_titre(self):
        cours = Cours(
            titre="Python",
            slug="python",
            resume="Un résumé suffisamment long.",
            contenu="Du contenu.",
            matiere=self.matiere,
        )
        self.assertEqual(str(cours), "Python")

    def test_url_absolue_utilise_le_slug(self):
        cours = Cours.objects.create(
            titre="Python",
            slug="python",
            resume="Un résumé suffisamment long.",
            contenu="Du contenu.",
            matiere=self.matiere,
        )
        self.assertEqual(cours.get_absolute_url(), "/cours/python/")

```

`setUpTestData` crée les données une fois pour la classe, ce qui est souvent plus rapide que `setUp`, exécuté avant chaque méthode. Utilisez `setUp` si le test doit modifier son jeu de données sans contaminer les autres tests.

Le test ne suppose pas que la table `cours_cours` existait déjà : Django applique les migrations à la base isolée. C'est une raison de tester les migrations sur une base vide plutôt que de compter sur votre SQLite locale.

Tester les URLs et les vues

`cours/tests/test_views.py` :

```

from django.test import TestCase
from django.urls import reverse

from cours.models import Cours, Matiere

class ListeCoursTests(TestCase):
    @classmethod
    def setUpTestData(cls):
        matiere = Matiere.objects.create(
            nom="Informatique",
            slug="informatique",
        )
        Cours.objects.create(
            titre="Cours visible",
            slug="visible",
            resume="Résumé visible.",
            contenu="Contenu visible.",
            matiere=matiere,
            statut=Cours.Statut.PUBLIE,
        )

```

```
Cours.objects.create(
    titre="Cours caché",
    slug="cache",
    resume="Résumé interne.",
    contenu="Brouillon.",
    matiere=matiere,
    statut=Cours.Statut.BROUILLON,
)

def test_liste_repond_200(self):
    response = self.client.get(reverse("cours:liste"))

    self.assertEqual(response.status_code, 200)
    self.assertTemplateUsed(response, "cours/liste.html")
    self.assertContains(response, "Cours visible")
    self.assertNotContains(response, "Cours caché")

def test_detail_inconnu_renvoie_404(self):
    response = self.client.get(
        reverse("cours:detail", kwargs={"slug": "inexistant"})
    )

    self.assertEqual(response.status_code, 404)
```

`self.client` simule un navigateur sans lancer de serveur. `reverse` fait apparaître les erreurs de nom de route immédiatement. `assertNotContains` est important ici : un filtre qui afficherait aussi les brouillons serait une régression fonctionnelle et potentiellement une fuite d'information.

Tester l'authentification et les permissions

```
from django.contrib.auth import get_user_model

User = get_user_model()

class CreationCoursTests(TestCase):
    def setUp(self):
        self.user = User.objects.create_user(
            username="alice",
            password="mot-de-passe-de-test",
        )

    def test_visiteur_redirige_vers_connexion(self):
        response = self.client.get(reverse("cours:creer"))

        self.assertEqual(response.status_code, 302)
        self.assertIn("/connexion/", response.url)

    def test_utilisateur_connecte_peut_afficher_le_formulaire(self):
        self.client.force_login(self.user)

        response = self.client.get(reverse("cours:creer"))

        self.assertEqual(response.status_code, 200)
        self.assertContains(response, "Créer un cours")
```

`force_login` évite de tester le formulaire de connexion en même temps que la vue de création. Un test doit isoler le comportement qu'il décrit.

Tester un formulaire

```
from django.test import TestCase

from cours.forms import CoursForm
from cours.models import Matiere

class CoursFormTests(TestCase):
    def setUp(self):
        self.matiere = Matiere.objects.create(
            nom="Python",
            slug="python",
        )

    def test_titre_trop_court_invalide_le_formulaire(self):
        form = CoursForm(
            data={
                "titre": "Py",
                "slug": "py",
                "resume": "Un résumé.",
                "contenu": "Contenu.",
                "matiere": self.matiere.pk,
            }
        )

        self.assertFalse(form.is_valid())
        self.assertIn("titre", form.errors)
```

Le test appelle `is_valid()` avant de consulter `errors`. Les valeurs de `ForeignKey` transmises à un formulaire HTTP sont des identifiants, pas des instances Python.

Déboguer sans deviner

Quand un test échoue, lisez le premier échec et son traceback. Puis répondez aux questions suivantes :

1. l'échec est-il dans l'arrangement des données, l'action, ou l'assertion ?
2. l'exception appartient-elle au code de production ou au test ?
3. le comportement attendu a-t-il changé volontairement ?
4. un autre test échoue-t-il à cause d'un état partagé ?

Les tests Django isolent la base, mais pas forcément les variables globales ou les fichiers que votre code écrit sur disque. Évitez les effets de bord et utilisez des répertoires temporaires pour les tests de fichiers.

Pour observer les requêtes exécutées dans un test :

```
from django.test.utils import CaptureQueriesContext
from django.db import connection

with CaptureQueriesContext(connection) as queries:
    response = self.client.get(reverse("cours:liste"))
```

```
print(len(queries))
```

N'associez pas la qualité à un nombre arbitraire de requêtes : vérifiez que chaque requête est nécessaire et que le nombre ne croît pas avec le nombre d'objets.

Qualité automatique avec ruff

Ajoutez Ruff en dépendance de développement :

```
uv add --dev ruff
uv run ruff check .
uv run ruff format --check .
```

Vous pouvez corriger automatiquement le formatage :

```
uv run ruff format .
```

Un outil de qualité ne remplace pas les tests : il détecte surtout des problèmes de style, d'imports ou de constructions suspectes. Les tests ne remplacent pas non plus la revue de conception. Ils se complètent.

Une petite stratégie de test

Pour BiblioCours, commencez par les comportements risqués :

- les cours brouillons ne sont jamais visibles publiquement ;
- un slug inconnu renvoie 404 ;
- un visiteur ne peut pas créer ou supprimer ;
- les champs obligatoires sont refusés ;
- le formulaire valide redirige après sauvegarde ;
- la recherche ne provoque pas d'erreur avec une chaîne vide ;
- les migrations partent d'une base vide.

Les tests de modèle peuvent vérifier les propriétés locales. Les tests de vue vérifient la route, le statut, le template et les données affichées. Les tests d'intégration plus larges servent pour les parcours critiques. N'écrivez pas un test séparé pour chaque ligne de code ; écrivez-en pour les contrats qui doivent rester vrais.

Exercices

1. Ajoutez un test qui vérifie qu'un cours brouillon renvoie 404 sur sa page publique.
2. Ajoutez un test POST valide qui vérifie la redirection et l'objet créé.
3. Ajoutez un test POST invalide qui vérifie que la base ne contient aucun objet partiellement créé.
4. Mesurez le nombre de requêtes de la liste pour 1 puis 20 cours et corrigez une éventuelle croissance $N+1$.

Préparer la production et continuer l'apprentissage

Développement et production n'ont pas le même but

Le serveur `runserver` accélère l'apprentissage : il recharge le code, montre les tracebacks et sert certains fichiers statiques. Il n'est pas conçu pour recevoir du trafic public. En production, l'application est exécutée derrière un serveur applicatif et souvent un proxy qui gère TLS, compression, fichiers statiques et limites réseau.

Le déploiement n'est pas une commande magique. Il consiste à rendre reproductibles plusieurs éléments :

- la version de Python ;
- les dépendances ;
- les réglages et secrets ;
- le schéma de base ;
- les fichiers statiques ;
- le processus qui sert Django ;
- les sauvegardes et la supervision.

Le guide de déploiement Django rappelle que la configuration par défaut est adaptée au développement et doit être audité avant exposition publique ([Django Software Foundation 2025d](#)). Une démarche de déploiement progressive et documentée est également au cœur des ouvrages consacrés à Django en production ([Hajian 2022](#)).

Séparer la configuration du code

Une base minimale de réglages utilise l'environnement :

```
# config/settings.py
import os
from pathlib import Path

BASE_DIR = Path(__file__).resolve().parent.parent

SECRET_KEY = os.environ["DJANGO_SECRET_KEY"]
DEBUG = os.environ.get("DJANGO_DEBUG", "false").lower() == "true"

ALLOWED_HOSTS = [
    value.strip()
    for value in os.environ.get("DJANGO_ALLOWED_HOSTS", "").split(",")
    if value.strip()
]
```

En développement, vous pouvez fournir des valeurs temporaires dans le shell :

```
export DJANGO_SECRET_KEY="cle-locale-uniquement"
export DJANGO_DEBUG="true"
export DJANGO_ALLOWED_HOSTS="127.0.0.1,localhost"
uv run python manage.py runserver
```

Préparer la production et continuer l'apprentissage

Ne copiez pas ces valeurs dans `settings.py` si le dépôt peut devenir public. Le système de secrets de l'hébergeur est le bon emplacement en production.

Les réglages de sécurité courants comprennent :

```
SECURE_SSL_REDIRECT = True
SESSION_COOKIE_SECURE = True
CSRF_COOKIE_SECURE = True
SECURE_HSTS_SECONDS = 31_536_000
SECURE_HSTS_INCLUDE_SUBDOMAINS = True
SECURE_CONTENT_TYPE_NOSNIFF = True
```

N'activez HSTS qu'après avoir vérifié que tout le domaine fonctionne en HTTPS : une mauvaise configuration peut rendre un domaine difficile à récupérer dans un navigateur. Les options doivent être adaptées à votre proxy et à votre hébergeur.

Exécuter les vérifications Django

Avant chaque déploiement :

```
uv run python manage.py check --deploy
uv run python manage.py test
uv run ruff check .
uv run python manage.py makemigrations --check --dry-run
```

La dernière commande échoue si des modifications de modèles ne sont pas accompagnées d'une migration. Elle évite de découvrir au moment du déploiement que le schéma de production est en retard.

Base de données et migrations

SQLite est excellente pour un premier projet et un apprentissage local. Une application multi-utilisateur en production utilisera souvent PostgreSQL ou une base gérée par l'hébergeur. Le modèle Django reste similaire, mais les opérations, les performances, les sauvegardes et les droits diffèrent.

Le processus de migration est :

```
uv run python manage.py migrate
```

Il doit être exécuté comme étape contrôlée du déploiement. Ne supprimez pas la base de production pour « repartir proprement » : une migration est une évolution du schéma, et la conservation des données fait partie du problème.

Une sauvegarde n'est utile que si sa restauration a été testée. Documentez :

- qui peut lancer une restauration ;
- quelle est la fréquence des sauvegardes ;
- quelle perte de données maximale est acceptable ;
- comment vérifier que l'application redémarre sur une copie restaurée.

Fichiers statiques

En production :

```
uv run python manage.py collectstatic --noinput
```

Le serveur applicatif ne devrait pas être responsable de servir efficacement chaque fichier CSS. Configurez un serveur web, un CDN ou un stockage objet pour `STATIC_ROOT`. Vérifiez ensuite qu'une page réelle charge le CSS avec le bon `STATIC_URL`.

Les médias envoyés par des utilisateurs nécessitent une stratégie séparée : stockage persistant, contrôle d'accès, limites de taille, politique de rétention et sauvegarde. Un disque local éphémère peut faire disparaître les fichiers à chaque redéploiement.

WSGI, ASGI et processus

Django peut être servi par WSGI pour les applications synchrones classiques :

```
uv add gunicorn
uv run gunicorn config.wsgi:application
```

ASGI permet aussi d'intégrer des usages asynchrones :

```
uv add uvicorn
uv run uvicorn config.asgi:application
```

Le choix du serveur et ses options dépendent de l'hébergeur et de la charge. Une fonction `async def` ne rend pas automatiquement toutes les opérations rapides : un appel bloquant à une base ou à une API peut toujours bloquer le worker. L'asynchronisme est une décision d'architecture, pas un mot-clé à ajouter partout.

Journalisation et erreurs

En production, désactivez les pages de debug mais conservez des journaux exploitables :

```
LOGGING = {
    "version": 1,
    "disable_existing_loggers": False,
    "handlers": {
        "console": {"class": "logging.StreamHandler"},
    },
    "loggers": {
        "django": {
            "handlers": ["console"],
            "level": "INFO",
        },
    },
}
```

Ne journalisez pas les mots de passe, tokens, cookies ou données personnelles inutiles. Un log est une copie de données qui peut vivre longtemps et être accessible à davantage de personnes que la base applicative.

Déployer avec un `pyproject.toml`

Le fichier doit déclarer les dépendances d'exécution. Dans un pipeline de déploiement, on peut utiliser :

```
uv sync --frozen
uv run python manage.py migrate
uv run python manage.py collectstatic --noinput
uv run python manage.py check --deploy
```

`--frozen` demande à `uv` de respecter `uv.lock` sans recalculer la résolution. Cela rend l'installation prévisible. Si `pyproject.toml` et `uv.lock` ne sont plus cohérents, le pipeline doit échouer : il ne faut pas réparer silencieusement la production en choisissant de nouvelles versions.

Une checklist finale

Avant de considérer BiblioCours comme publiable :

```
[ ] DEBUG est désactivé
[ ] SECRET_KEY et mots de passe viennent d'un gestionnaire de secrets
[ ] ALLOWED_HOSTS et CSRF_TRUSTED_ORIGINS sont corrects
[ ] HTTPS est actif
[ ] les migrations sont versionnées et appliquées
[ ] les fichiers statiques sont collectés et servis
[ ] les médias sont persistants et protégés
[ ] les tests passent dans un environnement propre
[ ] les logs ne contiennent pas de secrets
[ ] la base et les médias sont sauvegardés
[ ] une procédure de restauration est documentée
[ ] le serveur de développement n'est pas le serveur public
```

Cette liste ne remplace pas un audit. Elle fournit un point de départ concret pour ne pas oublier les réglages qui restent invisibles pendant le développement local.

Aller plus loin : une progression raisonnable

Une fois le catalogue maîtrisé, ajoutez une seule capacité à la fois :

1. **vues génériques et class-based views** pour réduire le code répétitif;
2. **comptes et profils** avec un modèle utilisateur adapté avant la première migration d'un nouveau projet ;
3. **recherche et filtres** avec pagination et indexation mesurées ;
4. **API JSON** avec Django REST Framework lorsque d'autres clients doivent consommer les données ;
5. **tâches asynchrones** pour les courriels et traitements longs ;
6. **observabilité** pour connaître les temps de réponse et les erreurs ;
7. **CI** pour exécuter tests, vérifications et migrations sur chaque modification.

Il est préférable de comprendre une petite application de bout en bout que de superposer très tôt dix bibliothèques dont on ne connaît pas les responsabilités. La documentation de Django, du langage Python et des outils utilisés doit accompagner cette progression ([Python Software Foundation 2025](#) ; [Encode 2025](#)).

Projet de synthèse

Terminez BiblioCours avec les exigences suivantes :

- une personne anonyme peut consulter les cours publiés ;
- un cours brouillon n'est jamais affiché sur une URL publique ;
- un éditeur authentifié peut créer et modifier un cours ;
- une matière ne peut pas être supprimée si elle contient des cours ;
- le formulaire affiche ses erreurs sans perdre les données valides ;
- les pages utilisent des URLs nommées et des templates hérités ;
- les tests couvrent le catalogue, les permissions et les formulaires ;
- `uv sync --frozen` permet de recréer l'environnement ;
- `check --deploy` n'affiche plus de problème connu non traité.

Pour chaque fonctionnalité, faites un commit qui raconte une décision :

```
git add .  
git commit -m "feat: protéger la création des cours"
```

Un historique lisible devient une documentation supplémentaire du projet. Il permet de comprendre non seulement l'état actuel, mais aussi la raison des évolutions.

Exercices de consolidation

1. Configurez un environnement de test séparé et faites tourner la suite depuis une base vide.
2. Simulez une variable manquante comme `DJANGO_SECRET_KEY` et rendez l'erreur compréhensible.
3. Écrivez une procédure de restauration locale à partir d'une sauvegarde SQLite de test.
4. Documentez dans le README les commandes `uv sync --frozen`, `migrate`, `collectstatic`, `check --deploy` et `test`.

References

- Astral. 2025. « Uv Documentation ». <https://docs.astral.sh/uv/>.
- Django Software Foundation. 2025a. « Cross Site Request Forgery Protection ». <https://docs.djangoproject.com/en/5.2/howto/csrf/>.
- Django Software Foundation. 2025b. « Django Documentation ». <https://docs.djangoproject.com/en/5.2/>.
- Django Software Foundation. 2025c. « Django Overview : Design Philosophies ». <https://docs.djangoproject.com/en/5.2/misc/design-philosophies/>.
- Django Software Foundation. 2025d. « How to Deploy Django ». <https://docs.djangoproject.com/en/5.2/howto/deployment/>.
- Django Software Foundation. 2025e. « Models ». <https://docs.djangoproject.com/en/5.2/topics/db/models/>.
- Django Software Foundation. 2025f. « Security in Django ». <https://docs.djangoproject.com/en/5.2/topics/security/>.
- Django Software Foundation. 2025g. « Testing in Django ». <https://docs.djangoproject.com/en/5.2/topics/testing/>.
- Django Software Foundation. 2025h. « The Django Template Language ». <https://docs.djangoproject.com/en/5.2/ref/templates/language/>.
- Django Software Foundation. 2025i. « Working with Forms ». <https://docs.djangoproject.com/en/5.2/topics/forms/>.
- Django Software Foundation. 2025j. « Writing Your First Django App ». <https://docs.djangoproject.com/en/5.2/intro/tutorial01/>.
- Encode. 2025. « Django REST Framework ». <https://www.django-rest-framework.org/>.
- Greenfeld, Daniel Roy, et Audrey Roy Greenfeld. 2017. *Two Scoops of Django : Best Practices for Django 1.11*. 1 éd. Two Scoops Press.
- Hajian, William S. Vincent. 2022. *Django for Professionals : Production Websites with Python and Django*. 4 éd. Independently published.
- OWASP Foundation. 2021. « OWASP Top 10 : 2021 ». <https://owasp.org/Top10/>.
- Percival, Harry, et Bob Gregory. 2024. *Test-Driven Development with Python*. 3 éd. O'Reilly Media.
- Python Software Foundation. 2025. « Python 3 Documentation ». <https://docs.python.org/3/>.